THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY

(Deemed to be University), Patiala

UCS503P – Project Proposal

NimbusVault

AI-Powered Cloud Storage & Document Intelligence Platform

Project Proposal Report

B.E./B.Tech. Computer Engineering, Third Year

Submitted by

1024030021 Saras

1024030020 Mehul Jain

1024030012 Mansimar Kaur

Submitted to

Dr. Paramveer Sidhu

Project Proposal Report

Department of Computer Science and Engineering

Thapar Institute of Engineering and Technology, Patiala

October 12, 2026

Contents

Abstract **2**

1 Introduction **3**

1.1 Problem Statement 3

1.2 Project Objectives 4

2 Proposed Solution **5**

2.0.1 Core Features 5

2.0.2 Sharing and Access Control 5

2.0.3 AI Document Intelligence 5

3 Methodology and Design **7**

3.0.1 Component Responsibilities 8

3.1 Core Workflow 8

3.2 Technical Stack 9

3.3 Database Design 9

3.4 AI Integration 9

3.4.1 Pipeline Description 10

4 Implementation and Evaluation **12**

4.1 Security Design 12

4.2 Development Methodology 12

4.3 Evaluation Criteria 13

4.4 Future Enhancements 13

4.5 Risks and Mitigation 14

5 Conclusion **15**

Abstract

NimbusVault is a cloud-based file storage and document intelligence platform developed to address the limitations of conventional local and legacy cloud storage systems. The design draws on established cloud storage architectures such as Google Drive and Dropbox, combining scalable object storage with a document understanding layer to provide secure file management along with intelligent search.

The system separates file storage from file metadata. Raw files are stored on Amazon S3, while metadata such as ownership, folder structure, permissions and file attributes is stored in a PostgreSQL database accessed through the Prisma ORM. File transfers use pre-signed URLs, so uploads and downloads happen directly between the browser and S3 without passing through the backend server. This keeps load off the backend and allows files of any size to be handled efficiently.

In addition to standard storage operations such as upload, download, folder organisation and file sharing, NimbusVault includes an AI-based document intelligence feature built around a Retrieval-Augmented Generation (RAG) pipeline. Uploaded documents are parsed, split into chunks and converted into vector embeddings, which are stored in a vector database. This lets users search their documents by meaning rather than by exact keywords, and ask questions about document content in natural language.

The platform uses JWT-based authentication, password hashing, AWS IAM policies and role-based access control to protect user data and shared resources. The backend is stateless and can run across multiple instances, so the system can scale horizontally as usage increases. Taken together, the project combines full-stack development, cloud infrastructure and applied AI in one system, and is intended to demonstrate practical engineering ability across each of these areas.

Chapter 1

Introduction

The amount of digital data generated by individuals and organisations has grown rapidly over the last decade, driven by high-resolution media, collaborative documents, and everyday digital records. This growth has exposed several limitations of traditional file storage, which was not built to handle data at this scale or to support the kind of remote, always-available access that most users now expect.

Storing files only on local devices, such as hard drives, USB drives or unmanaged network shares, comes with well-known problems. Storage capacity is limited, hardware can fail, and files kept on one device are not accessible from another. A single disk failure can mean permanent data loss. These limitations are part of why cloud storage has become the standard approach for personal and organisational data alike, since it removes the dependency on a single physical device and instead relies on distributed, redundant infrastructure.

Moving files to the cloud, however, does not solve every problem. As a user's collection of files grows, it becomes harder to locate a specific document through folder browsing or keyword search alone. Traditional storage systems understand file names, not the content of the files themselves, so users are left to remember roughly where or under what name something was saved.

Sharing is another area where common storage tools fall short. Users often need to share specific files with specific people for a limited period of time, without exposing their entire account, and this requires access-control features that many systems either do not offer or implement poorly.

NimbusVault is motivated by these three issues: the need for reliable, scalable cloud storage; the need for controlled and secure file sharing; and the need for search that understands document content rather than only file names. The project combines a conventional cloud storage architecture with an AI-based document intelligence layer, with the aim of showing how cloud infrastructure and applied AI can be combined within a single, practical system.

1.1 Problem Statement

The following problems are commonly observed in the way individuals and small teams store and manage files, even when cloud storage is already in use.

- Personal devices have limited storage capacity, and users are often forced to delete or move files simply to free up space.
- Files kept only on local devices are vulnerable to hardware failure, theft and accidental deletion, with no built-in redundancy.
- As the number of stored files grows, manually maintained folder structures become inconsistent and difficult to navigate.
- Keyword-based search only matches file names or indexed terms, so it performs poorly when a user cannot recall the exact name of a file.
- Existing storage systems have no understanding of what a document actually contains, so summarising or answering questions about a file requires opening and reading it manually.
- Sharing files securely is difficult: many tools either grant broad access to an entire account or rely on informal methods such as emailing attachments, with little control over expiry or permission level.

These issues together point to the need for a storage platform that is not only reliable and secure, but also able to understand the content it stores.

1.2 Project Objectives

1. Build a cloud-native storage platform that can handle files at scale with high availability.
2. Implement secure user authentication using standard protocols and password-handling practices.
3. Use AWS S3 for scalable, durable object storage of user files.
4. Keep file metadata (in PostgreSQL) separate from raw file objects (in AWS S3), so each layer can be scaled independently.
5. Implement pre-signed URL based file transfer, allowing uploads and downloads to go directly between the browser and S3.
6. Provide secure, permission-based file sharing with configurable access levels and link expiry.
7. Integrate an AI document assistant capable of semantic search and natural-language question answering over stored documents.

Chapter 2

Proposed Solution

NimbusVault addresses the problems described above through a full-stack cloud storage platform, organised into core storage features, sharing and access-control features, and an AI document intelligence layer.

2.0.1 Core Features

- User registration and login with JWT-based authentication
- File upload and download via pre-signed AWS S3 URLs
- Folder creation and management
- Search and filtering across files and folders
- In-browser preview for common file types
- Trash and restore for deleted files
- Storage analytics showing usage by file type, folder and time

2.0.2 Sharing and Access Control

- Shareable links with configurable permissions
- Expiring access for time-bound sharing
- Role-based access (owner, editor, viewer)
- File versioning to preserve and restore earlier versions

2.0.3 AI Document Intelligence

- Automated text extraction from uploaded documents
- Semantic search based on document meaning, not just keywords

- Natural-language question answering over document content
- A Retrieval-Augmented Generation (RAG) pipeline that grounds AI answers in the user's own files

Chapter 3

Methodology and Design

NimbusVault follows a layered architecture that separates the frontend, application backend, metadata storage, file storage and AI processing into distinct components. Figure 3.1 shows the overall flow of data between these layers.

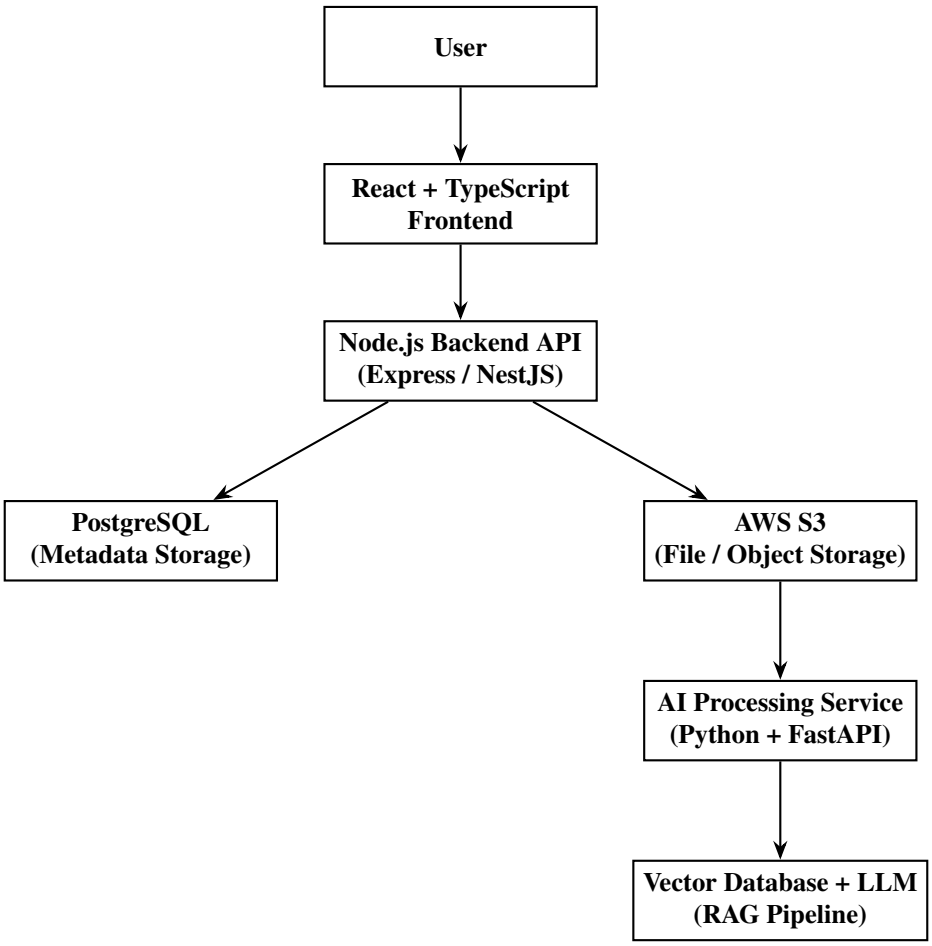


Figure 3.1: High-level system architecture of NimbusVault

3.0.1 Component Responsibilities

The frontend, built with React, TypeScript and Tailwind CSS, is responsible for rendering the interface, managing client-side state, and making authenticated API requests. It also handles file transfer directly to and from AWS S3 using pre-signed URLs issued by the backend, rather than routing file data through the backend server.

The backend, built with Node.js and Express or NestJS, exposes REST APIs for authentication, file and folder management, sharing, and search. It is responsible for generating pre-signed S3 URLs, enforcing authorisation rules, and coordinating requests to the AI processing service.

The database layer uses PostgreSQL with the Prisma ORM to store structured metadata, including users, folders, files, share links and permissions. Using an ORM keeps database access type-safe and easier to maintain as the schema evolves.

AWS S3 provides durable, scalable storage for the actual file content. It is kept fully separate from the metadata layer, with access controlled through AWS IAM policies and time-limited pre-signed URLs.

The AI service, written in Python using FastAPI, performs text extraction, chunking and embedding generation for uploaded documents, and handles semantic search and question-answering requests by querying the vector database and coordinating with the language model.

3.1 Core Workflow

The end-to-end workflow of NimbusVault, from account creation to secure file access, proceeds as follows:

1. A new user registers by providing basic credentials, which are validated and stored securely.
2. The user logs in; on success, the backend issues a signed JWT access token.
3. The user creates one or more folders to organise their files.
4. The user selects a local file to upload to a given folder.
5. The frontend requests a pre-signed upload URL from the backend.
6. The backend validates the request, checks authorisation, and generates a time-limited pre-signed S3 URL.
7. The frontend uploads the file directly to AWS S3 using that URL, without routing it through the backend.
8. Once the upload succeeds, the backend stores the file's metadata (name, size, type, S3 key, owner, folder) in PostgreSQL.
9. The file is now available for search, preview and download.
10. The owner may generate a secure sharing link with a chosen permission level and expiry.

11. When the shared link is accessed, the backend checks permissions and expiry before granting access to the file.

3.2 Technical Stack

Table 3.1 summarises the technologies proposed for each layer of NimbusVault.

Table 3.1: Proposed technology stack for NimbusVault

Layer	Technologies
Frontend	React.js, TypeScript, Tailwind CSS, React Router, React Query
Backend	Node.js, Express.js / NestJS, REST APIs, JWT, bcrypt
Database	PostgreSQL, Prisma ORM
Cloud	AWS S3, AWS EC2, AWS IAM, AWS CloudFront
DevOps	Docker, Docker Compose, GitHub Actions, CI/CD
AI / ML	Python, FastAPI, NLP, Embedding Models, Vector Database, RAG Pipeline, LLM Integration

3.3 Database Design

The metadata layer of NimbusVault is organised around four core entities, described in Tables 3.2 through 3.5.

Field	Description
`id`	Unique identifier for the user
`name`	Display name of the user
`email`	Unique email address used for login
`passwordHash`	Salted hash of the user's password

Table 3.2: `User` entity

3.4 AI Integration

NimbusVault's document intelligence feature is built around a Retrieval-Augmented Generation (RAG) pipeline, which grounds the language model's answers in the actual content of a user's documents rather than relying only on the model's general knowledge. Figure 3.2 shows the pipeline from document upload to query response.

Field	Description
`id`	Unique identifier for the file
`filename`	Original name of the uploaded file
`ownerId`	Foreign key referencing the owning `User`
`folderId`	Foreign key referencing the parent `Folder`
`s3Key`	Object key locating the file within AWS S3
`size`	File size in bytes
`mimeType`	MIME type of the stored file

Table 3.3: `File` entity

Field	Description
`id`	Unique identifier for the folder
`name`	Display name of the folder
`ownerId`	Foreign key referencing the owning `User`
`parentFolderId`	Self-referencing key enabling nested folder hierarchies

Table 3.4: `Folder` entity

Field	Description
`id`	Unique identifier for the share link
`fileId`	Foreign key referencing the shared `File`
`token`	Randomly generated access token embedded in the share URL
`permission`	Access level granted, for example view-only or edit
`expiry`	Timestamp after which the link becomes invalid

Table 3.5: `ShareLink` entity

3.4.1 Pipeline Description

When a document is uploaded, its text is first extracted from formats such as PDF, DOCX or plain text. The extracted text is then split into smaller, semantically coherent chunks, since embedding an entire document at once would lose the fine-grained context needed for accurate retrieval. Each chunk is passed through a pre-trained embedding model to produce a vector representation of its meaning, and these vectors are stored in a vector database for efficient similarity search.

When a user submits a query, either as a search term or a natural-language question, the same embedding model converts it into a vector, which is compared against the stored document vectors to find the most relevant chunks. For search, these chunks are returned directly. For question answering, the retrieved chunks are passed as context to a language model, which generates an answer grounded in the user's own documents rather than in the model's unrelated general knowledge. This keeps answers traceable back to the source material and reduces the risk of the model inventing information that is not actually

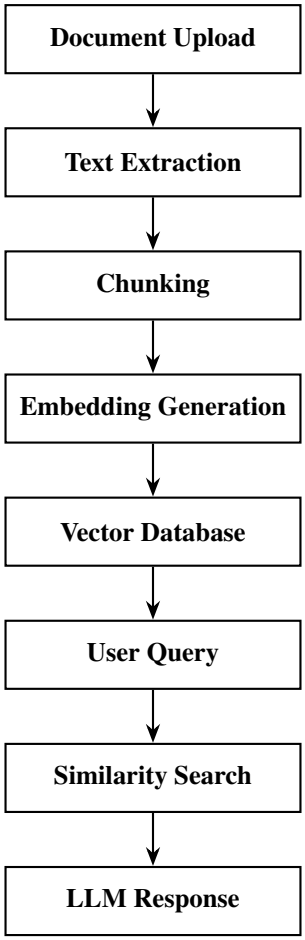


Figure 3.2: Retrieval-Augmented Generation (RAG) pipeline used for document intelligence

present in the documents.

Chapter 4

Implementation and Evaluation

NimbusVault's architecture is designed with horizontal scalability in mind, rather than treating it as an afterthought.

Storing file content in AWS S3 separately from metadata in PostgreSQL means each layer can be scaled according to its own load pattern; a spike in file uploads, for example, does not directly affect database load. Pre-signed URLs let file transfer bypass the application backend entirely, so the backend is not a bottleneck for large or frequent transfers. The backend itself is stateless, meaning it does not store session data locally, which allows multiple backend instances to run behind a load balancer without any need for session affinity. Deployment on AWS EC2, or an equivalent managed compute service, with auto-scaling allows capacity to adjust automatically as demand changes. Finally, computationally expensive tasks such as text extraction, chunking and embedding generation are handled by background workers coordinated through a message queue, so these tasks do not block or slow down the main API.

4.1 Security Design

Security is addressed at several layers of the platform. All authenticated requests require a valid, signed JWT, checked on every API call, and passwords are never stored in plain text; they are hashed using a strong, salted algorithm such as bcrypt before being saved. On the cloud side, AWS IAM policies restrict the application's permissions on S3 to the minimum required for it to function, and pre-signed URLs are both time-limited and scoped to a specific object and operation, so they cannot be reused indefinitely or for unrelated files. All communication between client and server is encrypted using HTTPS. Shared resources are protected by role-based access control, with owner, editor and viewer roles determining what actions a given user can perform, and sharing links use randomly generated tokens with configurable expiry to limit how long shared content remains accessible.

4.2 Development Methodology

The project will be developed in four phases, each building on the previous one to progressively add core, collaborative, operational and intelligent capabilities.

Phase	Focus Area	Key Deliverables
Phase 1	MVP Cloud Storage	Authentication, upload/download, folder management, basic search
Phase 2	Security & Collaboration	Secure sharing links, role-based access control, expiring permissions
Phase 3	Cloud Deployment & DevOps	Containerisation, CI/CD pipeline, cloud deployment, monitoring
Phase 4	AI Document Intelligence	Text extraction, embeddings, vector database, RAG-based Q&A

Table 4.1: Phase-wise development methodology

4.3 Evaluation Criteria

The completed system will be assessed against the following measurable criteria.

Metric	Description / Target
Upload/download success rate	Percentage of file transfers completed without error, targeting near-100% reliability
API response time	Average latency of core backend endpoints under typical load
Search accuracy	Relevance of results returned for keyword and semantic search queries
AI response relevance	Correctness and grounding of RAG-generated answers, checked against source documents
System reliability	Uptime and error rate of the deployed platform during evaluation
Security validation	Results of authentication, authorisation and access-control testing

Table 4.2: Evaluation metrics for the completed system

4.4 Future Enhancements

A number of extensions fall outside the current scope but are worth pursuing in future iterations of the project:

- Zero-knowledge encryption for end-to-end confidential file storage
- A native mobile application for iOS and Android

- Team workspaces with organisation-level administration
- More detailed usage and behavioural analytics
- Kubernetes-based deployment for better orchestration and resilience
- Multi-region storage replication for improved durability and lower latency

4.5 Risks and Mitigation

Risk	Mitigation Strategy
Large file handling	Direct browser-to-S3 upload via pre-signed URLs avoids routing large payloads through the backend
Unauthorized access	Layered checks using JWT authentication, AWS IAM policies and explicit permission checks
Incorrect AI-generated answers	RAG pipeline grounds responses in retrieved source documents, with citations back to the source content
Scope creep	Phase-based development with clearly defined deliverables for each phase

Table 4.3: Identified risks and corresponding mitigation strategies

Chapter 5

Conclusion

NimbusVault brings together full-stack web development, cloud infrastructure and applied AI within a single project. The storage architecture, based on separating metadata from file storage and transferring files directly through pre-signed URLs, reflects a practical, workable approach to building a scalable system rather than a purely theoretical one. The security measures used, including JWT authentication, password hashing, IAM policies and role-based access control, are standard practices in production systems and are applied here at a scope appropriate for an academic project.

The RAG-based document intelligence feature is what distinguishes NimbusVault from a simple storage clone, and its inclusion is intended to demonstrate familiarity with applied AI techniques such as embeddings, vector search and retrieval-grounded question answering. The phased development plan set out in this proposal is intended to keep the project achievable within the available time while still covering each of the major components described above.